
How Much Thinking Is Enough?

A Single-GPU AIME Speedrun

Aayush

Abstract

I tested reasoning limits, early verification and serving changes to reduce time to 18 with VibeThinker-3B on one A100. Initial gains weakened under repetition, which motivated a simpler prompt change. After selection on AIME 2024, it reduced time to 18 by 5.3% and 19.7% on two paired AIME 2022 seeds, with the same correct counts. The evidence supports those gains, but not yet a dependable improvement.

1 Approach

I kept the model fixed and judged changes by time to 18, including finalisation and grading. All scored runs used one A100 PCIe 80GB. Model startup was excluded; timing started before question loading. The experiments below show how failures and repeat runs changed the next decision.

2 From a working baseline to competing hypotheses

The first failure was truncation before a usable answer. A short finalisation request raised the initial AIME 2025 correct count from 8 to 16 at an 8,192-token cap. Increasing the cap to 12,288 reached 22 correct, with time to 18 of 199.9 s. I used this as a controlled starting point without assuming that 12,288 was optimal. I then tested whether the remaining cost came from excess reasoning, delayed feedback or the serving system.

Hypothesis and test	Observation	Resulting decision
Less reasoning costs less	A 10,240-token cap reached 18 in 181.1/181.4 s, but ended at exactly 18 in both runs.	Use 12,288 as the baseline for further improvements; change only the prompt in this test.
Check before continuing	Priced checks reached 14 sooner but 18 later in the first test.	Measure the full curve and investigate check overhead.
Reuse earlier processing	Prefix caching cut median finalisation from 5.88 to 0.36 s, but slowed the full run.	Keep the baseline because the full run was slower.
Overlap or reschedule work	Overlap gained 10.8% then lost 7.6%; async serving gained 8.7% then lost 1.3%.	Require repeated gains before changing the server.
Spread or stage reasoning	Two 6k samples reached only 15 correct twice; staged continuation lost both tests.	Retain the single-sample baseline.

Table 1. Experiments grouped by the question they tested rather than the order in which they ran. Serving results use matched controls for each change; they are not a single ranking. Historical traces remain in the local archive.

3 Can an early check save work?

I tested an early checkpoint at 8,192 tokens. A correct early answer stops the problem; otherwise reasoning resumes towards 12,288. The priced policy checks when estimated success probability times remaining reasoning time exceeds the estimated verification delay. It learns from checks within the run, not a trained predictor. Producing the early answer is already paid before the gate, and shared GPU work complicates the time estimate.

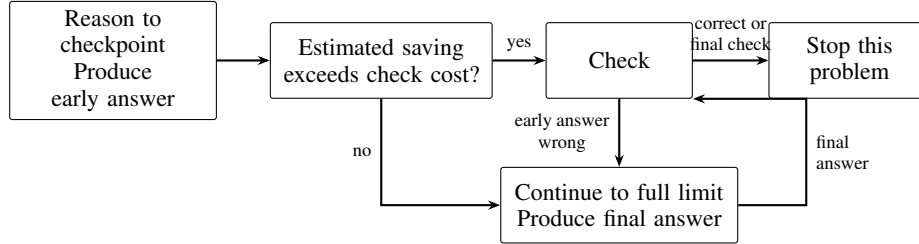


Figure 1. The continuation returns to the grader for a final check. There is only one early checkpoint; after the final check, the attempt ends.

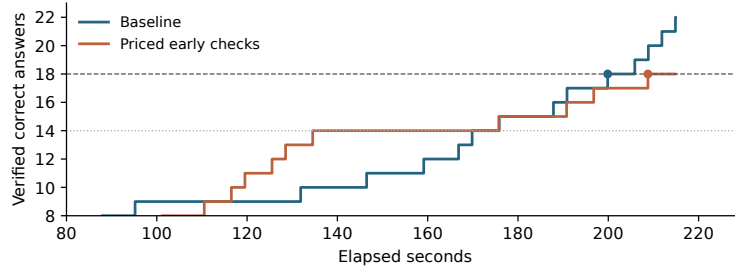


Figure 2. AIME 2025, seed 42. Priced checks reach 14 at 134.5 s versus 169.8 s, but reach 18 at 208.8 s versus 199.9 s. Each step is a positive verdict. The time axis is cropped to show the change in the lead.

The priced run made 33 checks rather than 29 and processed 2.28 times as much input text. These costs helped explain why an early lead could disappear, although this comparison does not isolate their individual effects.

Later, priced checks gained 12.4% on two AIME 2023 selection seeds. Six new pairs, with matched question orders, alternating execution order and fresh starts, gave four wins and two losses. Mean gain fell to 2.5%, with a descriptive 95% bootstrap interval of $[-3.44, 8.36]\%$. All twelve runs reached 18. The repeat changed my conclusion from a promising candidate to an uncertain gain.

4 Changing the instructions

The mixed results motivated a change that added no runtime machinery. I fixed three prompt variants before an AIME 2024 screen. With the same model and settings, the original reached 18 in 168.1 s; one clear method took 158.2 s, progress checks 157.6 s and focus on the requested quantity 111.3 s. Correct counts were 23, 25, 23 and 24 respectively. I selected the fastest variant reaching 18 and beating the original. GEPA motivated this small comparison; I did not run its full search or train the model. I used the development results to choose a prompt, then tested it separately.

5 Testing on separate problems

I fixed the selected prompt before exporting the AIME 2022 test questions, keeping these problems separate from those used to choose the prompt. I then ran two comparisons with seeds 811 and 813, without making further prompt changes.

Within each pair, both systems used the same question order and model settings. I restarted the model and grader before every run and reversed which system ran first in the second pair. This kept the comparison as consistent as possible.

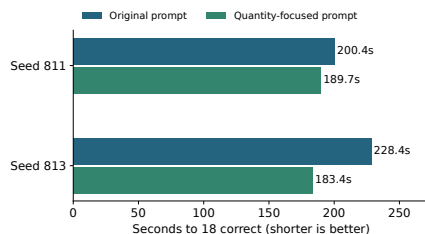


Figure 3. The selected prompt reached 18 correct answers sooner in both comparisons. Shorter bars mean faster runs.

Seed	Prompt	14 correct	16 correct	18 correct	20 correct	Total
811	Original	167.3	191.4	200.4	–	19
811	Selected	140.2	165.7	189.7	–	19
813	Original	204.4	216.4	228.4	–	18
813	Selected	117.1	130.3	183.4	–	18

Table 2. Times are in seconds. A dash means the system did not reach that number of correct answers. Total is the final correct count.

The selected prompt was 5.3% faster in the first comparison and 19.7% faster in the second, with the same correct count as the original within each pair. It also produced less text, which is consistent with doing less unnecessary work, although this does not establish why it was faster.

6 What the results show

The result comes from only two comparisons on one year. Both systems reached exactly 18 in the second pair, so neither had an accuracy margin. Separate problems also do not rule out exposure during the model’s original training. I would next freeze the prompt and test more seeds and another unused problem set, recording target failures alongside times. The earlier experiments show why a faster individual step or an early lead is insufficient on its own.

7 Code and time spent

Code, setup instructions and all eight prompt-trial records are at <https://github.com/ap-logics/aime-speedrun>. Their audits passed. Use `run.py answer-first` for the selected prompt and `baseline` for the control, with question and output paths.

I estimate approximately four hours of active work across sessions on 5–9 September, excluding waiting and unattended runs. This retrospective estimate covers the initial VM access problems and university-GPU fallback, A100 setup, implementation, experiment review and write-up. I did not time each activity separately; the dates do not represent continuous working days.

Recorded experiment rounds took 16m32s for the serving screen, 56m11s for the six-pair early-check confirmation and 35m09s for the prompt comparison. These are elapsed run times, not active working time or the total GPU time. Some active work overlapped with runs. The dated log is in `docs/time-log.md`.